Муниципальное бюджетное общеобразовательное учреждение

«Средняя школа №23»

ИНДИВИДУАЛЬНЫЙ ИТОГОВЫЙ ПРОЕКТ

Название проекта: «Разработка распределённой высоконагруженной платформы мгновенного обмена данными с поддержкой сквозного криптографического шифрования и горизонтального масштабирования»

Автор проекта:
Петросян Артём Максимович,
ученик 11 «Б класса»

Руководитель проекта:
ФИО ,
Информатика и ИКТ

2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПЛАТФОРМ ДЛЯ ОБМЕНА ДАННЫМИ И ЗАЩИТЫ ИНФОРМАЦИИ.....5
1.1. Отличия между синхронным и асинхронным программированием.....5
1.2. Принцип работы сетевой технологии WebSocket в реальном времени....7
1.3. Теория защиты информации: алгоритмы сквозного шифрования данных. 9
1.4. Проектирование веб-интерфейсов взаимодействия: технологии REST API и механизмы безопасности CORS.....11
1.5. Концепция безопасной авторизации на базе JWT-токенов (Access/Refresh).....13
1.6. Обзор серверных технологий: язык Go, фреймворк Fiber и СУБД PostgreSQL 15
1.7. Особенности разработки интерфейсов: библиотека React JS и компонентный подход.....18
1.8.Назначение кэширования в Redis и обратного проксирования через Nginx 20
1.9. Технологии мультимедийной связи: организация видеозвонков через WebRTC.....22

ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РАЗВЕРТЫВАНИЕ ВЕБ-МЕССЕНДЖЕРА «ШАРИНГАН» 25
2.1. Создание асинхронной серверной части мессенджера на языке Go..... 25
2.2. Реализация механизмов безопасности API: конфигурация CORS, Rate Limiting и middleware авторизации по JWT-токенам.....28
2.3. Программная реализация шифрования сообщений по стандарту AES-256-GCM 21.....31

2.4. Интеграция базы данных PostgreSQL для хранения истории чатов.....33
2.5. Настройка оперативного кэша Redis для ускорения работы сокетов.....35
2.6. Разработка пользовательского интерфейса и логики чата на React JS….37
2.7. Реализация потокобезопасного WebRTC-сигналинга в WebSocket-цикле 40
2.8. Организация раздачи медиафайлов и аватарок через веб-сервер Nginx. .42
2.9. Перспективы развития проекта: интеграция ИИ-ассистента и Web-Push уведомлений..... 44
ЗАКЛЮЧЕНИЕ.....47
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....49
ПРИЛОЖЕНИЯ.....51

ВВЕДЕНИЕ

Актуальность темы. В эпоху цифровизации и перехода на удалённые форматы коммуникации веб-мессенджеры стали основным инструментом как для личного, так и для корпоративного общения. Однако большинство популярных коммерческих платформ (таких как Telegram, WhatsApp или VK) имеют закрытый исходный код, хранят данные пользователей на сторонних серверах и подвержены рискам утечки персональной информации. В условиях необходимости обеспечения цифрового суверенитета и конфиденциальности создание собственных, независимых и защищённых платформ для обмена данными становится критически важной задачей. Настоящий проект посвящён разработке суверенного веб-мессенджера «Шаринган», сочетающего в себе высокую скорость работы и современные криптографические стандарты.

Проблема исследования заключается в существовании технологического противоречия: большинство простых учебных или корпоративных чатов строятся на синхронной архитектуре и хранят данные в открытом (не зашифрованном) виде. Это приводит к двум ключевым проблемам: во-первых, сервер быстро прекращает стабильную работу при росте числа одновременных подключений; во-вторых, любой злоумышленник или администратор базы данных может беспрепятственно прочитать чужую переписку.

Цель проекта — проектирование и программная реализация высокопроизводительного асинхронного веб-мессенджера «Шаринган» с поддержкой постоянного двустороннего соединения, сквозного шифрования сообщений и защиты сетевого API.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Изучить теоретические аспекты асинхронного программирования и особенности протокола **WebSocket** в сравнении с классическим **HTTP-протоколом**.
2. Исследовать принципы работы современных механизмов сетевой безопасности **(CORS, JWT-авторизация)** и криптографического стандарта **AES-256-GCM**.
3. Разработать асинхронную серверную часть (бэкенд) мессенджера на языке программирования **Go** с использованием фреймворка **Fiber**.
4. Спроектировать структуру базы данных **PostgreSQL** для хранения истории чатов и интегрировать кэш **Redis** для оптимизации обработки сокетов и **WebRTC-сигналинга**.
5. Создать клиентскую часть (фронтенд) мессенджера на базе библиотеки **React JS** с использованием компонентного подхода.
6. Настроить веб-сервер **Nginx** в качестве обратного прокси для безопасной и быстрой раздачи статических медиафайлов и аватарок пользователей.

Объект исследования: технологии и протоколы передачи данных в распределённых веб-системах реального времени.

Практическая значимость проекта заключается в том, что разработанный мессенджер «Шаринган» является полностью готовым программным продуктом (MVP). Он может использоваться в качестве локальной изолированной сети общения для образовательного учреждения или малого предприятия, гарантируя, что конфиденциальная переписка не

выйдет за пределы целевого сервера.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ СЕТЕВЫХ СИСТЕМ РЕАЛЬНОГО ВРЕМЕНИ И ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

1.1 Отличия между синхронным и асинхронным программированием

Синхронное программирование: Здесь все задачи выполняются строго по очереди. Компьютер не начнёт делать вторую задачу, пока полностью не завершит первую. Если сервер чата работает синхронно, то когда один пользователь загружает тяжёлую картинку, весь сервер «замирает». Остальные пользователи в этот момент не могут даже отправить текстовое сообщение — они ждут своей очереди. Для высоконагруженных систем это превращается в катастрофу.

Асинхронное программирование: Этот подход позволяет выполнять задачи параллельно. Если бэкенд-сервер запускает долгий процесс (например, запись сообщения в базу данных **PostgreSQL** или чтение файла), он не ждёт его окончания. Сервер мгновенно переключается на обслуживание запросов других пользователей, а к первой задаче возвращается тогда, когда она сама завершится в фоне. Именно асинхронность позволяет мессенджеру стабильно работать, даже когда в сети одновременно общаются сотни учеников.

1.2 Принцип работы сетевой технологии **WebSocket** в реальном

времени

Обычные сайты работают по протоколу **HTTP (принцип «запрос-ответ»)**: браузер просит данные, сервер отдаёт их и сразу закрывает связь. Для мессенджера это неудобно, ведь сервер не может сам прислать тебе новое сообщение, пока ты не обновишь страницу.

Технология **WebSocket** решает эту проблему:
1. Браузер один раз отправляет специальный **HTTP-запрос** с просьбой «переключить» связь.
2. Происходит «рукопожатие» **(Handshake)**, и между браузером и сервером открывается постоянный прямой туннель связи.
3. этот туннель работает в полнодуплексном режиме: и сервер, и клиент **могут в** любую секунду отправлять данные друг другу. Как только твой собеседник нажимает «Отправить», сервер тут же перекидывает сообщение в твой туннель. Чат работает мгновенно и без перезагрузок страниц.

1.3 Теория защиты информации: алгоритмы сквозного шифрования данных

Если хранить переписку в базе данных в виде обычного текста, её сможет прочитать любой злоумышленник или системный администратор, получивший доступ к серверу. Для защиты личных данных в проекте используется сквозное шифрование стандарта **AES-256-GCM**.

Симмитричный алгоритм **AES-256** признан криптографическим стандартом на военном уровне во всём мире. Приставка GCM означает, что алгоритм не просто шифрует текст, но и защищает его от подмены. Перед сохранением в

PostgreSQL сервер на **Go** превращает понятные слова в зашифрованные **Hex-строки** (не читаемый хлам). Обратно в текст этот хлам превращается только на устройстве получателя (дешифруется в текст). Если хакер перехватит пакет в сети или украдет базу данных, он увидит бесмысленный набор символов.

1.4 Проектирование веб-интерфейсов взаимодействия: технологии **REST API** и механизмы безопасности **CORS**

Для обмена стандартными данными (регистрации, авторизации, загрузка аватарок, отправка жалоб) **используется** архитектура REST API. Это набор понятных правил, по которым фронтенд (React) **общается** с бекендом **(Go)** с помощью обычных **HTTP-запросов (GET, POST, PUT, DELETE)**. Например, отправка формы регистрации — это строго **POST-запрос** на адрес **/api/register.**

Защищает эти роуты механизм **CORS (Cross-Origin Resource Sharing)**. По умолчанию современные браузеры запрещают сайтам делать запросы на чужие серверы. На бэкенде мессенджера настроена строгая **middleware-фильтрация CORS**: сервер разрешает принимать запросы исключительно от нашего официального сайта мессенджера, блокируя любые попытки хакеров отправить скрыть запрос со сторонних вредоносных ресурсов.

1.5 Концепция безопасной авторизации на базе **JWT-токенов** (**Access/Refresh**)

Вместо устаревшего хранения сессий в куках браузера, в «Шаринган» используеться современная авторизация на базе **JWT-токенов** (цифровых

паспортов). Когда ты вводишь правильный пароль, сервер генерирует и выдаёт тебе два зашифрованных токена:
Access Token (Токен доступа): Имеет короткий срок жизни (24 часа). Браузер сохраняет его в памяти и автомотически прикрепляет к каждому запросу в заголовке **Authorization**. Сервер считывает его и сразу понимает, какой именно пользователь к нему обращаеться. При этом пароль в сеть повторно не отправляется.

Refresh Token (Токен обновления): Имеет долгий срок жизни (30 дней) и храниться в базе кэша **Redis**. Когда короткий **Access-токен** сгорает, фронтенд втайне от пользователя отправляет **Refresh-токен** на сервер, получает новый **Access-токен** и сессия продолжаеться без вылетов и без повторного ввода пароля.

1.6 Обзор серверных технологий: язык **Go**, фреймворк **Fiber** и СУБД **PostgreSQL**

Архитектура серверной части проекта собрана из передовых технологий:

Язык Go (Golang): Это компилируемый язык от компании Google. Он создавался специально для высоких сетевых нагрузок. Благодаря поддержке горутины (супер-лёгких потоков), **Go** тратит в разы меньше оперативной памяти, чем **Node.js** или **Python**, и работает со скорость программ на **C++**. А главное у него простой читаемый синтаксис и по сложности изучения он как и python простой.

Фреймворк Fiber: Это веб-каркас для Go, который берет на себя всю внутреннюю маршрутизацию запросов. Он построен на сверхбыстром

движке **FastHTP**, что позволяет серверу мессенджера отвечать на запросы за считанные миллисекунды.

СУБД PostgreSQL: Мощная реляционная база данных. Она отвечает за строгое и надёжное зранение структуры таблиц: связей пользователей, чёрных списков, архивов и истории жало модераторам.

1.7 Особенности разработки интерфейсов: библиотека **React js** и компонентный подход

Клиентская часть (интерфейс который видит пользователь) написана на библиотеке **React js**. React использует революционный компонентный подход.
Это значит, что весь интерфейс мессенджера не пишется одним огромным куском кода, а собирается как конструктор LEGO из маленьких независимых деталей (компонентов): компонент формы ввода **AuthPage**, компонент списка чатов, компонент модального окна настроек. Каждый компонент имеет своё изолированное состояние **(useState)**. Когда тебе приходит новое сообщение, **React** точечно перерисовывает только маленькое окошко этого сообщения не перезагружая всю страницу целиком, что делает интерфейс невероятно плавным и отзывчивым.

1.8 Назначение кэширования в **Redis** и обратного проксирования через **Nginx**

Чтобы разгрузить и снизить нагрузку на сервер и сделать систему стабильной при пикивых нагрузках, внедрены два системных инструмента:

Кэш Redis: Это база данных, которая работает не с жёстким диском, а на прямую с оперативной памятью компьютера. Чтение из неё происходит мгновенно. Сервер кэширует в Redis последние 50 сообщений каждого чата, текущие статусы пользователей («Online»/«Offline») и сессионные токены. Это спасает основную СУБД PostgreSQL от перегрузки.

Прокси Nginx: Веб-сервер Nginx выступает в роли «щита» (обратного прокси) перед нашим Go-приложением. Он принимает на себя все внешние сетевые удары, занимается безопасной и быстрой раздачей статических файлов (аватарок пользователей, загруженных картинок и документов из папки uploads), освобождая **Go-сервер** для чистой обработки бизнес-логики и сокетов.

1.9 Технологии мультимедийной связи: organization видеозвонков через **WebRTC**

Современный мессенджер невозможно представить без звонков. Для реализации аудио- и видеосвязи напрямую между браузерами используется технология **WebRTC (Web Real-Time Communication)**.

Особенность **WebRTC** заключается в том, что после настройки связи медиа-трафик (голос и видео) идёт напрямую от одного пользователя к другому **P2P(Peer-to-Pear)**, минуя сервер мессенджера. Это полность снимает нагрузку с серверного процессора. Однако, чтобы два браузера нашли друг

друга в глобальной сети, им нужен посредник наш асинхронный **WebScoket-сервер** на **Go** как раз выполняет роль такого посредника **(Signaling Server)**. Он безопасно проверяет права пользователей, обходит чёрные списки и пересылает между ними технические сигналы связи **(offer, answer, ice-candidate)**, позволяя беспрепятственно установить прямое видео соединение.

ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РАЗВЕРТЫВАНИЕ ВЕБ-МЕССЕНДЖЕРА «ШАРИНГАН»

2.1 Создание асинхронной серверной части мессенджера на языке Go

Для создания серверной части (backend) части нашего месенджера был выбрал современный компилируемый язык программирования **Go (golang)**. Главными причинами его выбора стали его высокая скорось работы, низкое потребление оперативной памяти и встроенная поддержка эффективной много поточности с помощью легковесных потоков – **горутин (goroutines)**.

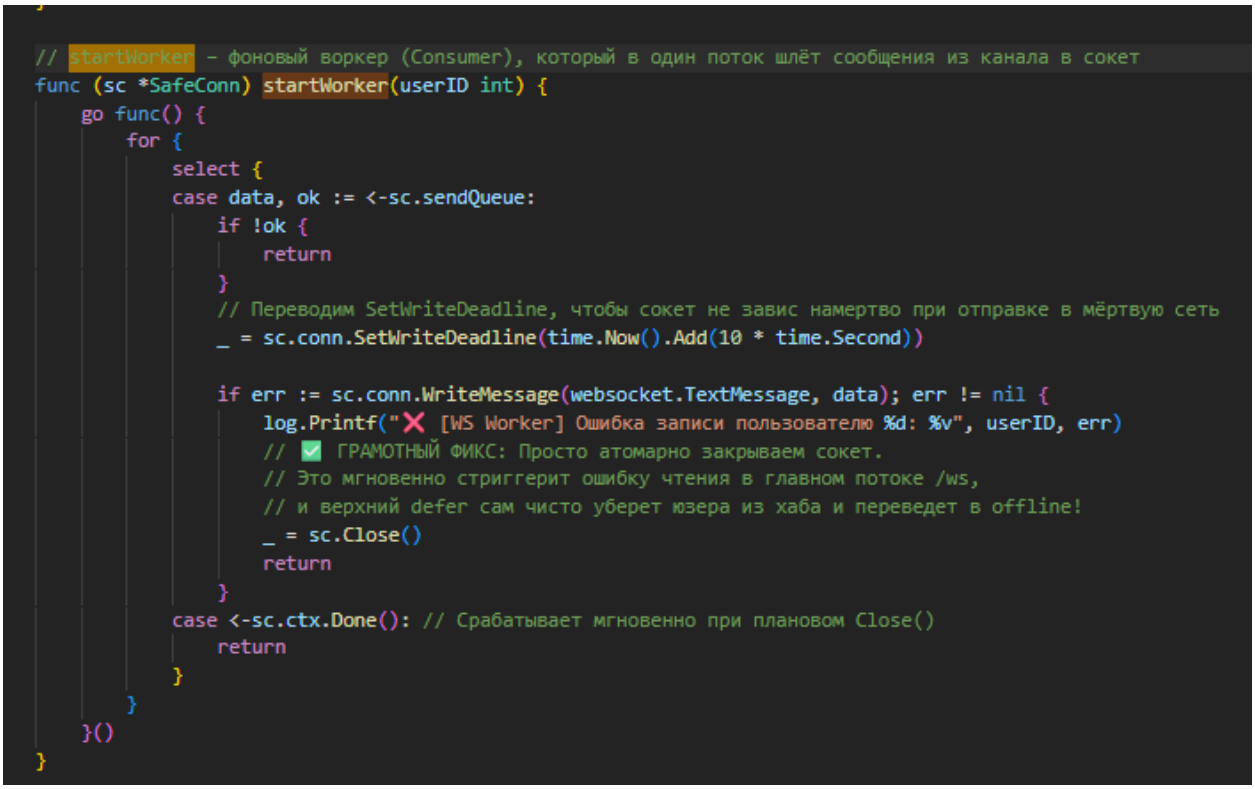
В качестве основы серверного веб-каркаса использован фреймворк **Fiber v2**.

Это современное решение, которое работает быстрее многих аналогов потому что под капотом используется оптимизированный движок обработки сетевых запросов. С его помощью в проекте развернуто полноценное **REST API** (программный интерфейс), обрабатывающее все стандартные действия пользователей: регестрацию, авторизацию, создание групповых чатов, поиск людей и управление контактами.

Особое внимание при проектировании сервера уделено стабильной и асинхронной работае сетевых сокетов **(WebSocket)**, через которые пользователи обмениваются мгновенными сообщениями. Чтобы сервер не зависал и не блокировался при одновременной отправке сообщений тысячам участников и более, была создана специальная программная структура **SaeConn**. Для каждого подключенного пользователя внутри этой структуры выделяется собственная изолированная очередь отправки в виде буферизированного канала **sendQueue**:

```
72
73  // SafeConn – обёртка над WebSocket соединением с изолированной очередью отправки
74  type SafeConn struct {
75      conn      *websocket.Conn
76      sendQueue chan []byte     // 🔥 Очередь отправки сообщений (наш канал)
77      ctx       context.Context // Контекст для контроля жизненного цикла воркера
78      cancel    context.CancelFunc
79      mu        sync.Mutex // Мьютекс оставляем для потокобезопасного Close
80  }
81
```

Когда на сервер поступает новое сообщение, оно не отправляется в сеть мгновенно (что могло бы вызвать задержки). Вместо этого она падает в индивидуальную очередь пользователя **sendQueue**. Параллельно на сервере запускается фоновый независимый поток-воркер **startWorker**,



```go
// startWorker - фоновый воркер (Consumer), который в один поток шлёт сообщения из канала в сокет
func (sc *SafeConn) startWorker(userID int) {
    go func() {
        for {
            select {
            case data, ok := <-sc.sendQueue:
                if !ok {
                    return
                }
                // Переводим SetWriteDeadline, чтобы сокет не завис намертво при отправке в мёртвую сеть
                _ = sc.conn.SetWriteDeadline(time.Now().Add(10 * time.Second))

                if err := sc.conn.WriteMessage(websocket.TextMessage, data); err != nil {
                    log.Printf("❌ [WS Worker] Ошибка записи пользователю %d: %v", userID, err)
                    // ✅ ГРАМОТНЫЙ ФИКС: Просто атомарно закрываем сокет.
                    // Это мгновенно стриггерит ошибку чтения в главном потоке /ws,
                    // и верхний defer сам чисто уберет юзера из хаба и переведет в offline!
                    _ = sc.Close()
                    return
                }
            case <-sc.ctx.Done(): // Срабатывает мгновенно при плановом Close()
                return
            }
        }
    }()
}
```

который в один поток забирает данные из этой очереди передает из в сокет клиенту. Есть у какого-то пользователя внезапно падает интернет, его очередь заполниться, и сервер автоматически и без болезненно разорвет зависшую сессию, не мешая работе остальных участников мессенджера.

2.2 Реализация механизмов безопасности **API**: конфигурация **CORS**, **RateLimiting** и **middleware** авторизации по **JWT-токенам**

Безопастность пользовательских данных и защита сервера от хакерских атак – важнейшая часть разработаки веб-мессенджера. Для комплексной защиты **API** были реализованны три основных рубежа обороны:

1. Конфигурация политики **CORS (Cross-Origin Resource Sharing)**. Данный механизм контролирует, какие веб-сайты имеют права отправлять запросы к нашему серверу. В коде сервера предусмотренно чтение переменной окружения **ALLOWED_ORIGINS**. Это гарантирует, что к приватным эндпоинтам мессенджера сможет обращаться только наш официальный доверенный фронтенд-сайт, а любые сторонние вредоносные скрипты из интернета будут заблокированы браузером на корню.

```
644     // Включаем middleware для перехвата всех HTTP-запросов к API
645     app.Use(prometheus.Middleware)
646
647     // ========== CORS (ИСПРАВЛЕНО: Защита от уязвимости CORS) ==========
648     allowedOrigins := os.Getenv("ALLOWED_ORIGINS")
649     if allowedOrigins == "" {
650         allowedOrigins = "*" // дефолтный fallback для локальной разработки
651     }
652     app.Use(cors.New(cors.Config{
653         AllowOrigins:     allowedOrigins,
654         AllowMethods:     "GET,POST,PUT,DELETE,OPTIONS",
655         AllowHeaders:     "Content-Type,Authorization,X-CSRF-Token",
656         AllowCredentials: true,
657     }))
658
659     // ========== Rate Limiting (ИСПРАВЛЕНО: Защита от паник nil pointer и float64) ==========
660     app.Use(limiter.New(limiter.Config{
661         Max:        250,
662         Expiration: 1 * time.Minute,
663         KeyGenerator: func(c *fiber.Ctx) string {
664             // Безопасное извлечение IP (защита от паники при nil)
665             ipLocal := c.Locals("real_client_ip")
666             clientIP, ok := ipLocal.(string)
667             if !ok {
668                 clientIP = c.IP()
669             }
670
671             userIDLocal := c.Locals("user_id")
672             if userIDLocal != nil {
673                 // Безопасный кастинг типов float64/int из JWT (защита от паники рантайма Go)
674                 var id int
675                 switch v := userIDLocal.(type) {
676                 case int:
677                     id = v
678                 case float64:
679                     id = int(v)
680                 default:
681                     return clientIP
682                 }
683                 return clientIP + ":" + strconv.Itoa(id)
684             }
685             return clientIP
686         },
687         LimitReached: func(c *fiber.Ctx) error {
688             return c.Status(429).JSON(APIResponse{
689                 Code:    429,
690                 Message: "Слишком много запросов, попробуйте позже",
691             })
692         },
693     }))
694
695     app.Use(logger.New())
696
```

2. Ограничение частоты запросов **(Rate Limiting)**. Чтобы зашитить сервер от **Ddos** атак (перезагрузка искусственными запросами) и спам-ботов,

подключен модуль **limitter**. Он настроен на лимит в 250 заросов в минуту с одного и того же источника. Программа динамически генерирует уникальный ключ для каждого проверяемого смартфона или компьютера, комбинируя реальный **IP-адресс** клиента и его личный ID из системы. Если лимит превышается, сервер временно выдаст ошибку под кодом 429 (Слишком много запросов).

3. Авторизация с помощью **JWT-токенов (JSON Web Tockents)**. Доступ к личным сообщениям и настройкам закрыт специальным промежуточным слоем — **authMiddleware**. При успешном входе в систему **/login** сервер выдает пользователю два зашифрованных токена: **access_token** (короткого действия для повседневных запросов) и **refresh_token** (длинного действия для обновления сессии). При каждом следующем действии приложение передает токен в заголовке **Authorization: <Bearer <токен>**. Программа расшифровывает токен спомошью секретного ключа **totpSecret**, проверяет срок его годности и извлекает ID пользователя. Дополнительно прямо внутри этого слоя внедрена проверка на глобальный бан аккаунта. Если модератор заблокировал нарушителя, **токен** мгновенно аннулируется, а пользователя выкидывает из системы.



```
        },
    })
})

authMiddleware := func(c *fiber.Ctx) error {
    authHeader := c.Get("Authorization")
    if len(authHeader) < 7 || authHeader[:7] != "Bearer " {
        return c.Status(401).JSON(fiber.Map{"error": "Неверный формат авторизации"})
    }
    tokenString := authHeader[7:]
    secret := []byte(os.Getenv("totpSecret"))
    token, err := jwt.Parse(tokenString, func(t *jwt.Token) (interface{}, error) {
        if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, fiber.ErrUnauthorized
        }
        return secret, nil
    })
    if err != nil || !token.Valid {
        return c.Status(401).JSON(fiber.Map{"error": "Неверный или просроченный токен"})
    }
    claims, ok := token.Claims.(jwt.MapClaims)
    if !ok {
        return c.Status(401).JSON(fiber.Map{"error": "Ошибка чтения токена"})
    }
    userIDFloat, ok := claims["user_id"].(float64)
    if !ok {
        return c.Status(401).JSON(fiber.Map{"error": "Ошибка в ID пользователя"})
    }

    userID := int(userIDFloat)

    // ИСПРАВЛЕНИЕ: Динамическая проверка бана прямо при каждом запросе (защита от "живого" JWT)
    var isBanned bool
    err = dbConn.QueryRow(
        context.Background(),
        "SELECT EXISTS(SELECT 1 FROM user_relations WHERE user_id=0 AND target_user_id=$1 AND relation_type='blocked
        userID,
    ).Scan(&isBanned)
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"error": "Ошибка проверки статуса аккаунта"})
    }
    if isBanned {
        return c.Status(403).JSON(APIResponse{Code: 403, Message: "🔴 Ваш аккаунт временно недоступен."})
    }

    c.Locals("user_id", userID)
    return c.Next()
}
```

2.3 Программная реализация шифрования сообщений по стандарту **AES-256-GCM**

Для защиты тайны переписки в мессенджере «Шаринган» реализовано надежное симметричное шифрование текство сообщений на стороне сервера перед из сохранением в базу данных. Для этого используется передовой криптографический стандарт **AES-256-GCM** в режиме **GCM (Galois/Counter Mode)**. Данный режим не просто кодирует текст, но и гарантирует его

целостность: если злоумышленник попытается изменить хотя бы один бит в зашифрованной строке внутри баззы данных, система выдаст ошибку при расшифровке.

За этот фнкционал отвечают две функции **encryptMessage (шифрование)** и **decryptMessage (расшифровка)**. Процесс устроен следующим образом:
1. Сервер берет секретный 32-байтный ключ из файла настроек сервера **MESSAGE_ENCRYPTION_KEY**.

2. С помощью системного генератора случайных чисел rand.Reader для каждого сообщения создается уникальный вектор инициализации – nonce (одноразвый код). Это нужно для того, чтобы два абсолютно одинаковых слова (например два «Привет») шифровались в совершенно разные наборы символов.

3. Метод **aesGCM.Seal** кодирует текст и прикрепляет nonce к началу получившейся строки результат переводиться в понятный для баз данных текстовый шеснадцатеричный фрмат (**Hex-строку**).

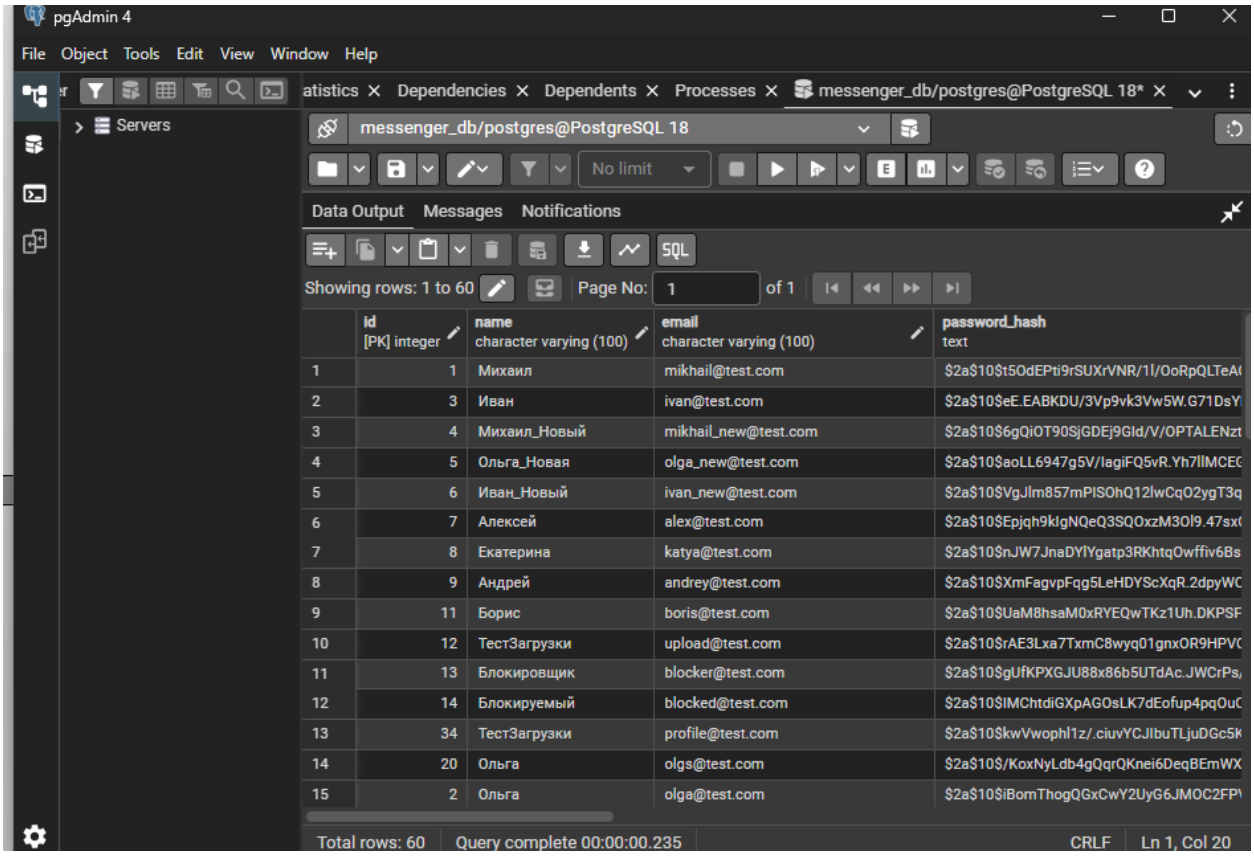
В результате в таблице истории сообщений вместо обычных слов храниться безопасная зашифрованная «абракадабра». При чтении чата функция **decryptMessage** считывает **Hex-строку**, отделяет от нее nonce, с помощью того же 32-байтного ключа производит **декодирование** и возвращает пользователям исходный понятный текст.

Пример такой строки

1e4184c41f05e2c1396c3554718961e2f35af5e09e3d37243a1c3c85d59a393cb0

2.4 Интеграция базы данных **PostgreSQL** для хранеия истории чатов

Основным постоянным хранилищем всей информации в проекте выступает мощная реляционная система управления базами данных (СУБД) **PostgreSQL**. Она отвечает за надежное долгосрочное хранение учетных записей пользователей, структуры созданных групп, информации о друзьях, поданных заявок на вступление и, главное, миллионов зашифрованных сообщений чатов. Для подключения сервера Go к **PostgreSQL** используеться продвинутый **драйвер pgx/v5** и технология пула соединений **pgxpool.Pool**. Вместо того чтобы открывать новое сетевое сооединение к базе данных на каждый чих пользователя (что сильно тормозило бы систему), сервер при старте создает заранее определенный набор готовых каналов связи. Когда роут API делает запрос к СУБД, он на миллисекунду одалживает свободное



соединение из пула, выполняет задачу и мгновенно возвращает его обратно. В коде реализованы строгие **SQL--запросы**, защишенные от инъекций благодаря плейсхолдерам **$1 $2** в коде. В качестве примера надежного проектирования данных можно выделить эндпоинта получения списка чатов **/chats**. Программа использует сложные обобщенные табличные выражения . С их помощью СУБД за один проход выбирает из базы данных все комнаты пользователя (личные диалоги, списки друзей и группы), находит для каждого чата ровно одно последнее отправленное сообщение, автоматически расшифровывает его и сортирует список комнат по времени последней активности.

2.5 Настройка оперативного кеша **Redis** для ускоренния работы сокетов

Даже самая быстрая база данных вроде **PostgreSQL** начнет замедляться, если сотни пользователей будут ежесекундно обновлять свои экранны и запрашивать историю переписки заного. Чтобы разгрузить основной сервер и обеспечить мессенджеру мгновенный отклик мы интегрировали быструю базу данных **Redis**, работающую прямо в оперативной памяти сервера.

Redis в нашем проекте выполняет две важнейшие функции:
1. Оперативное кэширование истории сообщений. Когда пользователь открывает групповой чат или любой другой чат, сервер сначала проверяет, нет ли готового списка сообщений в памяти **Redis** по ключу вроде **chat:group:ID**. Если данные есть, они мгновенно отдаются клиенту в обход жесткого диска. Если происходит отправка, редактирование или удаление сообщения, сервер производит автоматическую инвалидацию кэша – удаляет старый ключ из **Redis** (**redisClient.Client.Del**, заставляя систему обновить

данные при следующе запросе.

2. Горизонтальное масштабирование через шину **Redis Pub/Sub** (Издатель/Подписчик). Это решение позволяет запускать несколько копий нашего сервера мессенджера одновременно (например, в **Docker-контейнерах**). При старте сервер запускает фоновую горутину **startGlobalPubSubListener**, которая подписывается на шаблон каналов всех пользователей **ws:user:***. Когда сообщение отправляется в личный чат, оно публикуется в **Redis Pub/Sub. Redis** мгновенно разносит эту весть по всем копиям серверов бэкенда, и тот сервер, к которому физически подключен получатель в данный момент, доставляет ему сообщение по **WebSocket**. Это закладывает прочный фундамент для будущего масштабирования проекта на миллионы пользователей.

2.6 Разработка пользовательского интерфейса и логики чата на React js

Для создания визуальной (клиентской) части мессенджера «Шаринган» была выбрана библиотека **React JS**. Она позволяет строить интерфейс из независимых кусочков кода — компонентов (одно окно чата, одна карточка пользователя, одна кнопка), которые мгновенно перерисовываются на экране при изменении данных без перезагрузки всей страницы.

Логика взаимодействия фронтенда с нашим сервером построена на двух рельсах:
1. **HTTP-запросы (через стандартный fetch или библиотеку Axios):** используются для действий, требующих разового ответа от сервера — регистрация, авторизация **(/api/login)**, отправка тяжелых файлов на сервер **(/api/media/upload)** или редактирование своего профиля **(/api/profile)**. При

успешном входе **React** сохраняет полученный **access_token** в память приложения, чтобы автоматически прикреплять его к заголовкам каждого запроса.

2. **Постоянное WebSocket-соединение:** создается сразу после успешного входа пользователя в систему. **React** открывает соединение по адресу **/api/ws?token=<токен>**. Внутри кода фронтенда запускается слушатель событий **socket.onmessage**.

Когда с бэкенда прилетает пакет с типом **"new_message"** или **"delete_message"**, **React** подхватывает его и мгновенно обновляет массив сообщений в памяти браузера. Благодаря этому интерфейс работает плавно: новое сообщение тут же отображается на экране получателя, а удалённое или отредактированное — синхронно исчезает или меняет свой текст во всех открытых вкладках.

2.7. Реализация потокобезопасного WebRTC-сигналинга в WebSocket-цикле

Одной из самых технологичных функций мессенджера «Шаринган» стала поддержка аудио- и видеозвонков между пользователями напрямую по технологии **WebRTC (Web Real-Time Communication)**. Сама по себе технология **WebRTC** позволяет двум браузерам передавать голос и видео напрямую друг другу **(P2P)**, минуя наш сервер. Однако, чтобы созвон начался, браузеры должны сначала найти друг друга в сети и обменяться техническими данными — этот процесс называется **сигналингом**.

Для передачи этих служебных пакетов **(offer, answer, ice-candidate, hangup)** мы задействовали уже существующий **WebSocket-канал**. В бесконечном цикле чтения сообщений на сервере **Go** реализован специальный рубеж защиты **WebRTC-сигналинга**:

Когда пользователь А нажимает кнопку «Позвонить», его **React-приложение** генерирует событие **"call_signal"** и отправляет его в сокет. Сервер **Go** выступает в роли умного и безопасного почтальона: он моментально перехватывает этот пакет, делает супербыструю проверку по базе данных, не находится ли кто-то из участников в чёрном списке **(user_relations)**, подставляет честный **sender_id** (чтобы избежать подмены личности) и асинхронно пересылает пакет в сокет пользователю Б. Пользователь Б принимает сигнал, его браузер включает камеру, отправляет ответный сигнал, и между ними устанавливается прямое видеосоединение.

2.8. Организация раздачи медиафайлов и аватарок через веб-сервер Nginx

Современный мессенджер не может существовать без возможности обмениваться фотографиями, видеороликами, голосовыми сообщениями и документами. Для этого в системе развернут полноценный медиа-модуль.

Процесс отправки и скачивания файлов реализован максимально безопасно для сервера:

1. **Загрузка файлов (/api/media/upload):** на бэкенде установлен жесткий лимит на размер файла в **15 МБ** (защита от перегрузки диска). При

получении файла сервер полностью стирает его оригинальное имя (например, virus.exe) и заменяет его на случайный уникальный набор символов — **UUID** (например, 3f51a2-uuid...bin). Файл сохраняется в изолированную папку uploads на сервере и теряет возможность запуститься самостоятельно.

2. **Скачивание файлов (/api/media/download/:filename):** чтобы защитить систему от хакерских атак типа *Path Traversal* (когда злоумышленники пытаются выкачать секретные файлы сервера через подстановку путей вроде ../../etc/passwd), имя запрашиваемого файла жестко очищается функцией filepath.Base. Сервер принудительно отдаёт файл в режиме **Content-Type**: application/octet-stream и выставляет заголовок **Content-Disposition**: attachment. Это заставляет браузер скачивать файл на компьютер как безопасный документ, а не выполнять его код внутри мессенджера.

В продакшн-версии (при развертывании проекта) всю работу по хранению и быстрой отдаче этих картинок и файлов берет на себя веб-сервер **Nginx**. Он настраивается так, чтобы запросы к файлам в папке /uploads/ шли напрямую через него, минуя наш бэкенд на Go. Это освобождает оперативную память бэкенда для обработки более важных задач — пересылки текстовых сообщений и удержания сокетов.

2.9. Перспективы развития проекта: интеграция ИИ-ассистента и Web-Push уведомлений

Созданный мессенджер «Шаринган» имеет гибкую модульную архитектуру,

что позволяет легко расширять его функционал в будущем. В качестве главных перспектив развития проекта определены два направления:

Интеграция ИИ-ассистента: в мессенджер планируется внедрить умного чат-бота на базе языковой модели (например, через **API** Google Gemini). ИИ сможет выступать в роли личного помощника для пользователей: саммаризировать (кратко пересказывать) содержание длинных групповых чатов, переводить сообщения с иностранных языков прямо внутри диалога и помогать модераторам автоматически выявлять спам и токсичное поведение в публичных сообществах.

Реализация Web-Push уведомлений: чтобы пользователи не пропускали важные сообщения, когда вкладка с мессенджером закрыта, в бэкенд уже заложен фундамент для работы пуш-уведомлений по стандарту **RFC 8030**. В коде сервера созданы эндпоинты для сохранения криптографических подписок браузера **(/api/profile/webpush)** и подключена библиотека ://github.com. В функции **sendCustomWebPush** прописана логика: если пользователя нет в сети **WebSocket**, сервер берет его подписку из СУБД, шифрует JSON-пакет с именем отправителя и текстом сообщения с помощью приватного **VAPID-ключа** и отправляет его на шлюз уведомлений (например, серверы Google или Apple). Смартфон или компьютер пользователя получает пуш на экране даже при полностью выключенном браузере.